

Universidad Gerardo Barrios
Facultad de Ciencia y Tecnología



Informe Final del Trabajo de Graduación en Modalidad Pre-especialización – Módulo 4
“Implementación y escalabilidad”.

Nombre del Proyecto:

Gerardito - IA para recomendación de carreras universitarias.

Presentado por:

Fátima del Carmen Ayala Santos – SMIS003321

Fernando Rubén Chévez Sánchez – SMIS890919

Meylin Nohely Reyes Medina – SMIS032721

Docente:

Ing. Marco Antonio Arévalo Zambrano

San Miguel, 01 de agosto de 2026

Índice

Introducción	3
Resumen Ejecutivo	3
Descripción del problema, necesidad u oportunidad abordada.....	4
Objetivos Del Proyecto	4
Objetivo General:	4
Objetivos Específicos:.....	5
Justificación	5
Descripción de la solución desarrollada	6
Arquitectura de la Aplicación	6
Diagrama de Componentes, flujo de datos o arquitectura lógica.	8
Tecnologías, lenguajes, frameworks, librerías, modelos o APIs utilizadas	8
Descripción de los componentes de inteligencia artificial incorporados.....	9
Plan de infraestructura y costos iniciales.	10
Consideraciones de MLOps aplicadas o proyectadas	12
Estándares de calidad de software considerados.....	13
Evidencias del funcionamiento.	14
Análisis de resultados obtenidos.....	16
Limitaciones actuales de la solución.	16
Consideraciones éticas, de seguridad y uso responsable de la IA.	16
Conclusiones	17
REFERENCIAS.....	18

Introducción

La orientación vocacional constituye un proceso estratégico dentro de las instituciones de educación superior, ya que incide directamente en la toma de decisiones académicas de los estudiantes y, en consecuencia, en su permanencia, rendimiento y proyección profesional. Tradicionalmente, las universidades han implementado test vocacionales sustentados en sistemas basados en reglas, contruidos a partir del conocimiento experto de profesionales en psicología y educación.

Sin embargo, dichos sistemas presentan limitaciones inherentes a su naturaleza estática, ya que no incorporan mecanismos de aprendizaje a partir de datos históricos ni ajustan sus recomendaciones conforme evolucionan los patrones de elección académica de los estudiantes. Basados en este contexto, el presente anteproyecto propone el diseño e implementación de un sistema híbrido que combine un enfoque basado en reglas con técnicas de aprendizaje automático, con el propósito de optimizar la precisión y personalización de las recomendaciones vocacionales.

El proyecto se enmarca en el ámbito de la inteligencia artificial aplicada a sistemas de apoyo a la decisión, integrando métodos de clasificación supervisada y análisis de similitud. A lo largo del documento se desarrollan la justificación, los objetivos, el alcance y limitaciones, la descripción técnica de la solución propuesta, el esquema de validación de resultados, los recursos necesarios y el cronograma de ejecución.

Resumen Ejecutivo

"Gerardito", es un proyecto que continúa su evolución como un sistema inteligente de orientación vocacional para la Universidad Gerardo Barrios (UGB). Durante este módulo, la aplicación dio un paso importante al migrar de una arquitectura monolítica hacia una arquitectura desacoplada basada en una API REST desarrollada con FastAPI, permitiendo separar la lógica de negocio del componente de interfaz y facilitando la escalabilidad del sistema.

La solución mantiene como núcleo un Modelo de Lenguaje Grande (Llama 3.1) ejecutado localmente mediante Ollama e integrado con técnicas de Procesamiento de Lenguaje Natural

(NLP), permitiendo comprender los intereses y habilidades de los aspirantes para generar recomendaciones vocacionales personalizadas. Como parte de la evolución del proyecto, se incorporó validación estricta de datos utilizando esquemas Pydantic, mejorando la seguridad, consistencia y confiabilidad de la información procesada antes de ser enviada al modelo de inteligencia artificial.

De esta manera, se implementaron pruebas automatizadas con pytest y un proceso de integración continua mediante GitHub Actions, fortaleciendo la calidad del software y permitiendo validar automáticamente el correcto funcionamiento de los servicios desarrollados. Dichos avances establecen una base sólida para la siguiente etapa del proyecto, orientada a la integración completa del frontend, la persistencia de datos y el despliegue mediante contenedores

Descripción del problema, necesidad u oportunidad abordada

La adecuada elección de una carrera universitaria es un factor determinante en la trayectoria académica y profesional de los estudiantes. Diversos estudios evidencian que una elección inadecuada puede derivar en deserción temprana o cambios recurrentes de carrera, generando impactos personales e institucionales. Los sistemas de orientación actuales basados exclusivamente en reglas carecen de adaptación dinámica y tienen una limitada capacidad de personalización. Existe la necesidad de un sistema conversacional que permita a los estudiantes expresarse con sus propias palabras, tolerando variaciones del lenguaje (jerga local, faltas de ortografía) para cruzar sus perfiles con el catálogo oficial de la UGB de forma precisa y empática

Objetivos Del Proyecto

Objetivo General:

Diseñar, documentar y demostrar una aplicación inteligente de recomendación vocacional que se llama (Gerardito), que integre la oferta institucional con técnicas de procesamiento de lenguaje natural (NLP) e inteligencia artificial generativa con el fin de mejorar la precisión y calidad de las recomendaciones de carreras universitarias.

Objetivos Específicos:

1. Desarrollar flujos de validación semántica orientados a la clasificación de perfiles vocacionales ingresados de forma libre por los estudiantes, aplicando tolerancia a fallos ortográficos.
2. Implementar una arquitectura segura que integre un LLM local y herramientas NLP en una interfaz web funcional.
3. Analizar la calidad de la interacción mediante un módulo de análisis de sentimientos híbrido para procesar la retroalimentación final de los usuarios.

Justificación

La adecuada elección de una carrera universitaria es un factor determinante en la trayectoria académica y profesional de los estudiantes. Diversos estudios evidencian que una elección inadecuada puede derivar en una deserción temprana, bajo rendimiento académico o cambios recurrentes de carrera, generando impactos tanto personales como institucionales.

En la actualidad, los sistemas de orientación vocacional basados exclusivamente en reglas presentan las siguientes limitaciones:

- Falta de adaptación dinámica a nuevos patrones de comportamiento estudiantil.
- Ausencia de aprendizaje a partir de datos históricos institucionales.
- Limitada capacidad de personalización basada en evidencia empírica.

La incorporación de un chatbot impulsado por inteligencia artificial permite complementar el sistema tradicional mediante la comprensión y validación semántica de los intereses del usuario.

Un modelo conversacional posibilita integrar el conocimiento experto existente con la capacidad de análisis de interacciones en tiempo real.

Desde el punto de vista académico, el proyecto contribuye a la formación especializada mediante la aplicación práctica de conocimientos en:

- Inteligencia artificial generativa y Modelos de Lenguaje Grande (LLMs).

- Ingeniería de Prompts (Prompt Engineering) y seguridad de la información.
- Procesamiento de Lenguaje Natural (NLP) y análisis de sentimientos.
- Desarrollo de interfaces reactivas para sistemas de apoyo a la decisión.

Asimismo, la propuesta posee pertinencia institucional al plantear una solución escalable y replicable en el contexto universitario, orientada a fortalecer los procesos de acompañamiento académico.

Descripción de la solución desarrollada

"Gerardito" es una aplicación inteligente que apoya el proceso de orientación vocacional de los aspirantes a la Universidad Gerardo Barrios. El sistema permite ingresar habilidades e intereses mediante opciones predefinidas y texto libre para generar recomendaciones personalizadas de carreras. Durante este módulo, la aplicación evolucionó hacia una arquitectura basada en una API REST desarrollada con FastAPI, separando la lógica de negocio de la interfaz de usuario. También, se incorporó la validación de datos mediante Pydantic, garantizando que la información procesada sea correcta antes de ser enviada al modelo Llama 3.1, ejecutado localmente con Ollama. Asimismo, spaCy complementa el procesamiento mediante técnicas de Procesamiento de Lenguaje Natural (NLP), fortaleciendo el análisis de las interacciones de los usuarios.

Arquitectura de la Aplicación

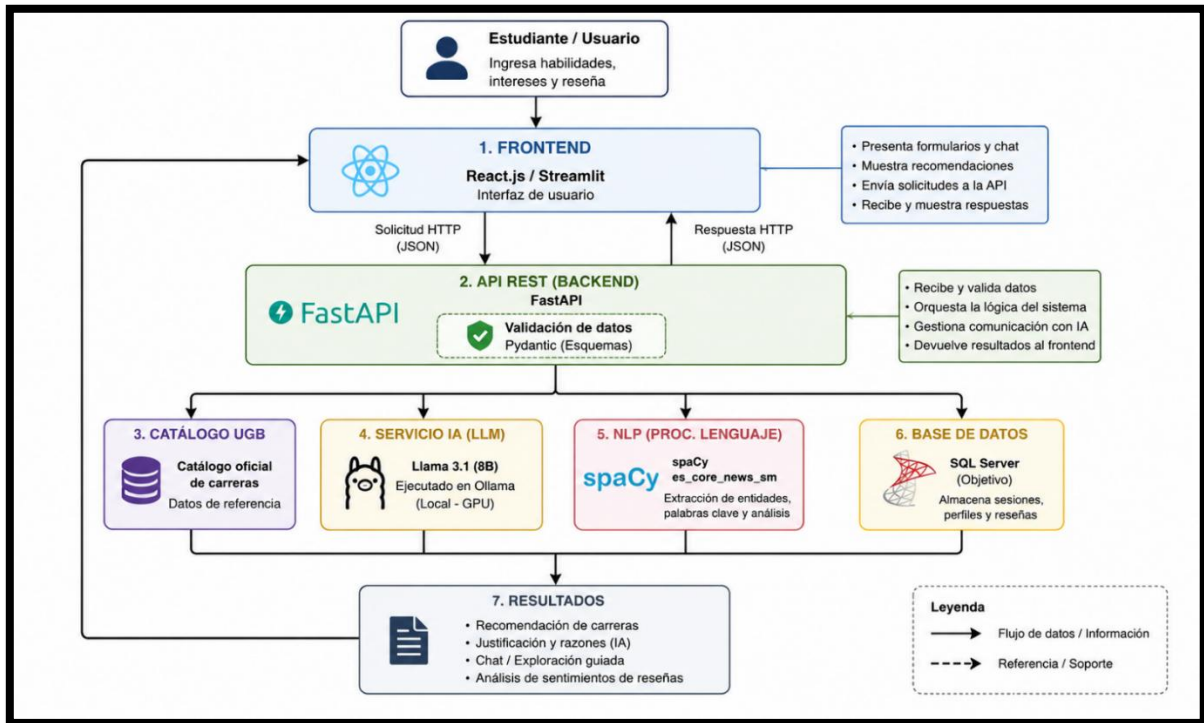
La solución integra una API REST desarrollada con FastAPI, encargada de gestionar la lógica del sistema, la validación de datos y la comunicación con los servicios de inteligencia artificial.

Con la siguiente tabla se describen los principales componentes que conforman la arquitectura actual de la aplicación.:

Componente	Tecnología	Descripción
Frontend	React.js / Streamlit	Interfaz encargada de la interacción con el usuario.

Backend	FastAPI	API REST que procesa las solicitudes y coordina la lógica del sistema.
Validación	Pydantic	Verifica y valida los datos recibidos antes del procesamiento.
Servicio IA	Ollama + Llama 3.1 + spaCy	Genera recomendaciones y procesa el lenguaje natural.
Base de Datos	SQL Server (objetivo)	Permitirá almacenar sesiones, recomendaciones y reseñas.
Despliegue	Docker (objetivo)	-Input del usuario (UI) -Filtro de Seguridad (Llm Validador) -Razonamiento y Cruce de Datos (LLM Diagnostico) -Salida de Interfaz Conversacional
Despliegue	Entorno Local (Windows/GPU)	Facilitará la ejecución y distribución de la aplicación.

Diagrama de Componentes, flujo de datos o arquitectura lógica.



Este diagrama muestra la arquitectura lógica de "Gerardito" y el flujo de información entre sus principales componentes. El proceso inicia cuando el usuario ingresa sus habilidades e intereses desde la interfaz de usuario, Estas solicitudes son enviadas a una API REST desarrollada con FastAPI. Posteriormente, los datos son validados mediante Pydantic y procesados por el servicio de inteligencia artificial, en el cual utiliza Llama 3.1 y spaCy para analizar el perfil del estudiante y generar recomendaciones vocacionales. Finalmente, se generan los resultados y son devueltos a la interfaz para su visualización en el frontend, permitiendo además la evaluación de reseñas mediante el análisis de sentimientos.

Tecnologías, lenguajes, frameworks, librerías, modelos o APIs utilizadas

Para el desarrollo de "Gerardito" se integran diferentes tecnologías y herramientas las cuales permiten implementar una solución inteligente, escalable y organizada.

En la siguiente tabla se presentan las principales tecnologías, lenguajes, frameworks, librerías, modelos y APIs utilizadas:

Categoría	Tecnologías, Lenguajes, Frameworks, Librerías, Modelos o APIs
Lenguaje de programación	Python 3
Frontend	Streamlit
Backend	FastAPI
Framework de validación	Pydantic
Modelo de Inteligencia Artificial	Llama 3.1 (8B)
Motor de IA	Ollama
Procesamiento de Lenguaje Natural	spaCy
Base de datos	SQL Server
Pruebas automatizadas	Pytest
Integración continua	GitHub Actions
Contenerización	Docker

Descripción de los componentes de inteligencia artificial incorporados

En este proyecto se incorporaron tres componentes inteligentes críticos:

- 1- **Filtro de Seguridad Semántico:** Utiliza Llama 3.1 para identificar intenciones válidas e inválidas en las solicitudes del usuario, permitiendo filtrar entradas irrelevantes o inapropiadas antes de generar una respuesta.
- 2- **Motor de Recomendación Vocacional:** Emplea Llama 3.1 junto con el catálogo de carreras de la Universidad Gerardo Barrios para analizar las habilidades e intereses del estudiante y generar recomendaciones personalizadas con una justificación basada en el perfil identificado.
- 3- **Analizador de Procesamiento de Lenguaje Natural:** Utiliza la librería **spaCy** para procesar el texto ingresado por los usuarios, identificar palabras clave y apoyar el análisis de las reseñas mediante técnicas de Procesamiento de Lenguaje Natural (NLP).

- 4- **Servicio de Integración mediante API REST:** Los componentes de inteligencia artificial se encuentran integrados a través de una **API REST desarrollada con FastAPI**, la cual valida las solicitudes mediante **Pydantic** antes de enviarlas al modelo de IA, garantizando una comunicación organizada y segura entre los diferentes componentes del sistema.

Plan de infraestructura y costos iniciales.

- Plan de infraestructura

Para el funcionamiento del sistema "**Gerardito**", se propone una infraestructura mínima, la cual, permita ejecutar de forma estable la aplicación, garantizando el procesamiento local del modelo de inteligencia artificial, la comunicación entre los diferentes componentes del sistema y la protección de la información de los usuarios.

Componente	Tecnología propuesta	Función
Cliente	Navegador web (React.js / Streamlit durante desarrollo)	Permite la interacción entre el usuario y el sistema.
Servidor de Aplicaciones	FastAPI (Python)	Procesa las solicitudes, valida los datos mediante Pydantic y coordina la comunicación con el servicio de IA.
Servicio de Inteligencia Artificial	Ollama + Llama 3.1 (8B) + spaCy	Genera las recomendaciones vocacionales y realiza el procesamiento del lenguaje natural.
Base de datos	SQL Server	Almacena la información de usuarios, recomendaciones y reseñas para futuras consultas y análisis.
Contenerización	Docker	Facilita el despliegue, aislamiento y mantenimiento de la aplicación en diferentes entornos.

- **Supuestos operativos**

Para la implementación inicial del proyecto se consideraron los siguientes supuestos:

- La aplicación será utilizada inicialmente dentro de la Universidad Gerardo Barrios como prototipo funcional.
- El procesamiento del modelo de inteligencia artificial se realizará de forma local mediante Ollama, evitando el envío de información a servicios externos.
- El servidor dispondrá de conexión permanente a la red institucional.
- El número estimado de usuarios concurrentes durante la etapa de pruebas será de entre 10 y 20 usuarios simultáneos.
- La base de datos almacenará las sesiones de los usuarios, recomendaciones generadas y reseñas para futuras mejoras del sistema.
- El despliegue se realizará inicialmente en un equipo institucional con sistema operativo Windows y posteriormente podrá migrarse a un servidor dedicado o máquina virtual.

- **Seguridad básica.**

Con el propósito de garantizar la disponibilidad e integridad del sistema, se consideran las siguientes medidas básicas de seguridad:

- Validación de todos los datos recibidos mediante esquemas Pydantic, evitando información inválida antes del procesamiento.
- Ejecución del modelo de inteligencia artificial en un entorno local mediante Ollama, preservando la privacidad de la información de los estudiantes.
- Restricción del tamaño de las entradas enviadas por los usuarios para prevenir sobrecarga del modelo.
- Separación entre la interfaz de usuario y la lógica del sistema mediante una API REST desarrollada con FastAPI.
- Uso de variables de entorno para la configuración de parámetros sensibles del sistema.
- Implementación de pruebas automatizadas con pytest e integración continua mediante GitHub Actions, permitiendo detectar errores antes de cada actualización del sistema.

- **Costos iniciales estimados.**

Recurso	Cantidad	Costo estimado
Equipo de cómputo con GPU dedicada (RTX 4060 o superior)	1	\$1,200.00
Sistema Operativo Windows	1	\$0.00 (licencia institucional)
Python	-	\$0.00
FastAPI	-	\$0.00
Ollama	-	\$0.00
Modelo Llama 3.1 (8B)	-	\$0.00
spaCy	-	\$0.00
Docker	-	\$0.00
GitHub Actions	-	\$0.00 (Plan gratuito)
SQL Server Express (desarrollo)	-	\$0.00

Por lo tanto, el costo estimado de implementación sería: \$1,200.00

- **Consideraciones finales.**

La infraestructura que se ha propuesto permite ejecutar el sistema de forma local, manteniendo la privacidad de la información procesada y reduciendo significativamente los costos operativos al emplear herramientas de código abierto. Además, la arquitectura desacoplada basada en FastAPI facilita la futura migración hacia una infraestructura institucional más robusta, incorporando servidores dedicados, bases de datos de mayor capacidad y contenedores Docker para soportar un mayor número de usuarios concurrentes.

Consideraciones de MLOps aplicadas o proyectadas

El ciclo de vida del modelo fue gestionado considerando las limitaciones y requerimientos del hardware disponible para garantizar un funcionamiento estable y eficiente.

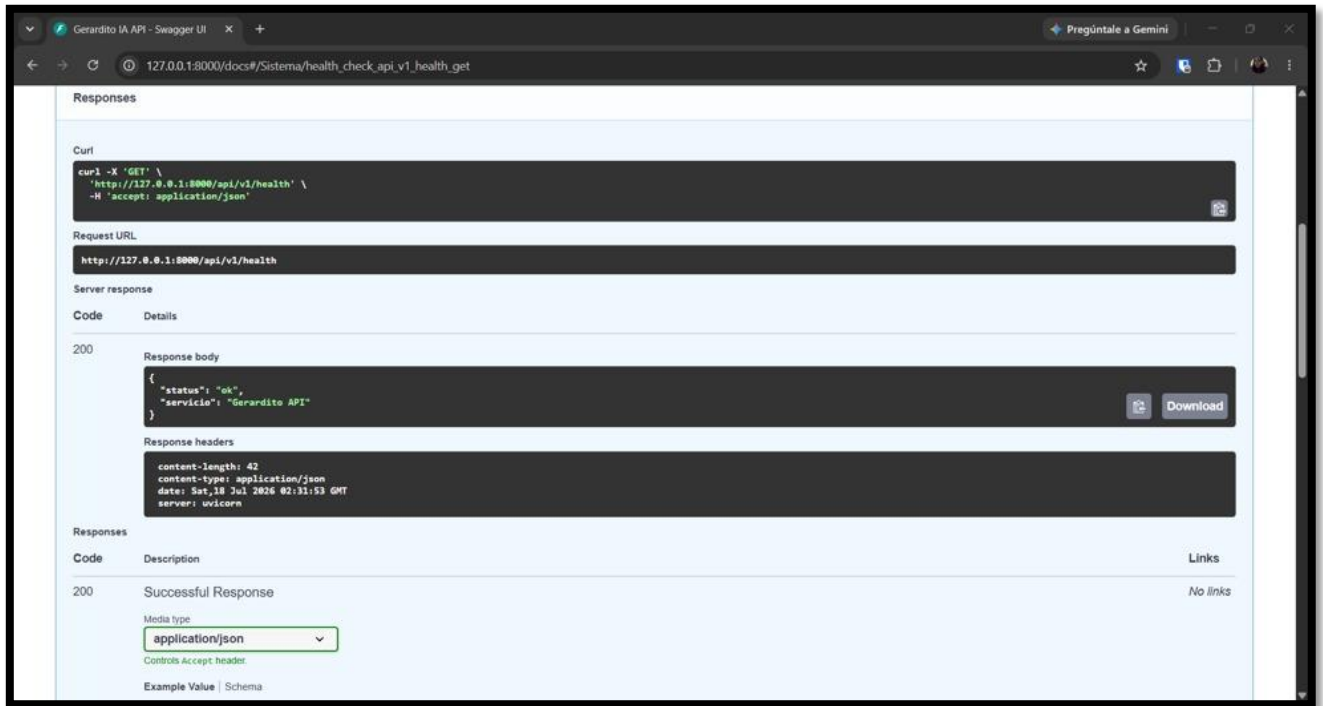
- **Gestión y Despliegue de Modelos:** Se utilizó Ollama como plataforma de inferencia local para administrar y ejecutar el modelo Llama 3.1, facilitando su configuración, reutilización y control dentro del entorno de desarrollo.

- **Optimización de Recursos Computacionales:** Se implementó el parámetro `keep_alive="24h"` durante la inicialización del modelo, para fijar los ~5 GB de pesos matemáticos en la VRAM de la GPU. Esto evita la latencia de descarga y recarga en memoria durante inactividad, asegurando respuestas en milisegundos durante demostraciones en vivo.
- **Control de Generación:** Se configuró el parámetro `num_predict=512` para garantizar tiempos de respuesta rápidos y evitar desbordamientos de buffer.

Estándares de calidad de software considerados.

- **Seguridad y Confiabilidad:** Se implementaron mecanismos de validación de datos para garantizar que la información procesada por el sistema cumpla con la estructura esperada, reduciendo errores y fortaleciendo la seguridad de la aplicación.
- **Mantenibilidad:** Se adoptó una arquitectura desacoplada que facilita la organización del código, la integración de nuevos componentes y el mantenimiento de la aplicación durante las siguientes etapas del proyecto
- **Calidad del Software:** Se incorporaron pruebas automatizadas y procesos de verificación continua para detectar errores de manera temprana y asegurar el correcto funcionamiento de los principales componentes del sistema.
- **Escalabilidad:** La arquitectura implementada permite la incorporación de nuevas funcionalidades y facilita el despliegue de la aplicación en entornos con mayores requerimientos, favoreciendo su crecimiento futuro.

Evidencias del funcionamiento.



Gerardito IA API - Swagger UI

127.0.0.1:8000/docs#/Sistema/health_check_api_v1_health_get

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/api/v1/health' \
  -H 'accept: application/json'
```

Request URL

http://127.0.0.1:8000/api/v1/health

Server response

Code	Details
200	Response body <pre>{ "status": "ok", "servicio": "Gerardito API" }</pre> Response headers <pre>content-length: 42 content-type: application/json date: Sat, 18 Jul 2026 02:31:53 GMT server: uvicorn</pre>

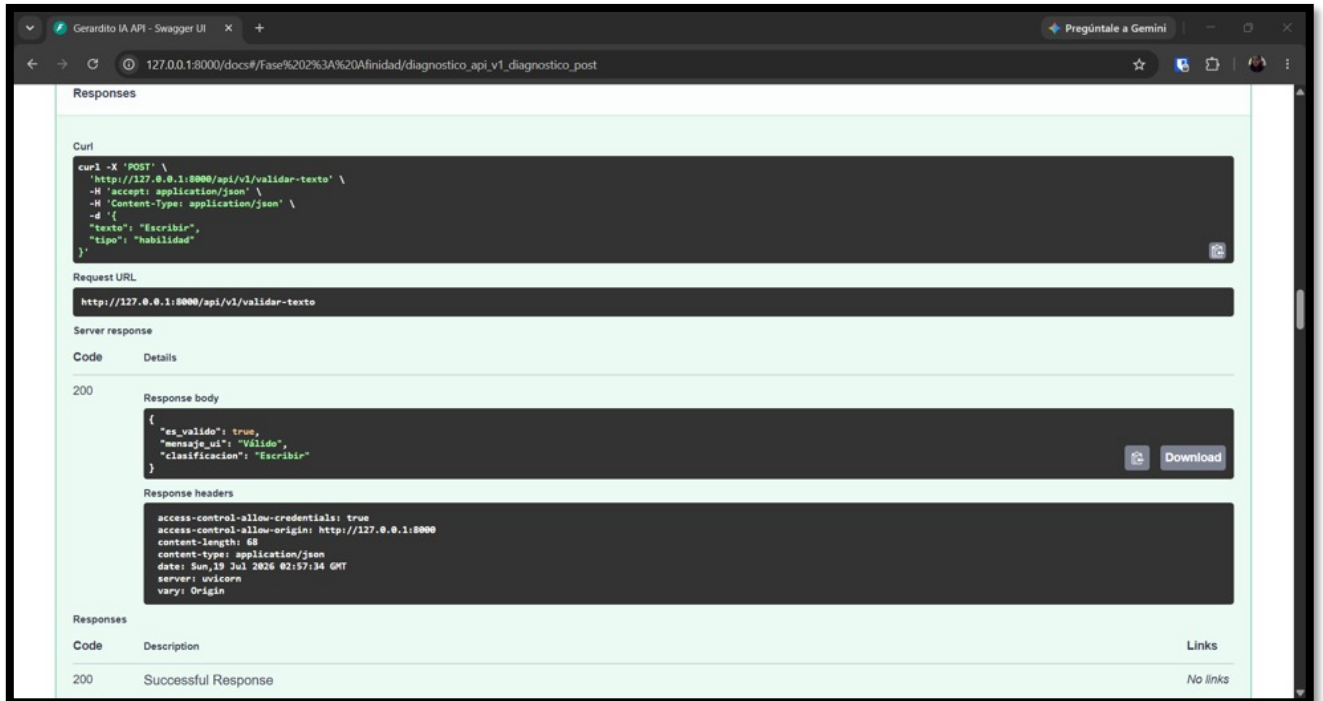
Responses

Code	Description	Links
200	Successful Response	No links

Media type: application/json

Controls Accept header

Example Value | Schema



Gerardito IA API - Swagger UI

127.0.0.1:8000/docs#/Fase%202%3A%20Afinidad/diagnostico_api_v1_diagnostico_post

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/api/v1/validar-texto' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "texto": "Escribir",
    "tipo": "habilidad"
  }'
```

Request URL

http://127.0.0.1:8000/api/v1/validar-texto

Server response

Code	Details
200	Response body <pre>{ "es_valido": true, "mensaje_ul": "Válido", "clasificacion": "Escribir" }</pre> Response headers <pre>access-control-allow-credentials: true access-control-allow-origin: http://127.0.0.1:8000 content-length: 68 content-type: application/json date: Sun, 19 Jul 2026 02:57:34 GMT server: uvicorn vary: Origin</pre>

Responses

Code	Description	Links
200	Successful Response	No links

Gerardito IA API - Swagger UI

127.0.0.1:8000/docs/#/Fase%201%3A%20Recolección

Pregúntale a Gemini

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/api/v1/resena' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "comentario": "Me encanto, es justo lo que buscaba"
  }'
```

Request URL

http://127.0.0.1:8000/api/v1/resena

Server response

Code	Details
200	Response body <pre>{ "mensaje": "¡Gracias por tu reseña! Ha sido procesada.", "sentimiento": "POSITIVO", "palabras_clave": ["justo"] }</pre> Response headers <pre>access-control-allow-credentials: true access-control-allow-origin: http://127.0.0.1:8000 content-length: 118 content-type: application/json date: Sun, 19 Jul 2025 04:06:00 GMT server: uvicorn vary: Origin</pre>

Responses

Code	Description	Links
200	Successful Response	No links

Gerardito IA API - Swagger UI

127.0.0.1:8000/docs/#/Fase%201%3A%20Recolección

Pregúntale a Gemini

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/api/v1/resena' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "comentario": "Me encanto, es justo lo que buscaba"
  }'
```

Request URL

http://127.0.0.1:8000/api/v1/resena

Server response

Code	Details
200	Response body <pre>{ "mensaje": "¡Gracias por tu reseña! Ha sido procesada.", "sentimiento": "POSITIVO", "palabras_clave": ["justo"] }</pre> Response headers <pre>access-control-allow-credentials: true access-control-allow-origin: http://127.0.0.1:8000 content-length: 118 content-type: application/json date: Sun, 19 Jul 2025 04:06:00 GMT server: uvicorn vary: Origin</pre>

Responses

Code	Description	Links
200	Successful Response	No links

Análisis de resultados obtenidos.

Los resultados obtenidos durante el Módulo 4 evidencian una evolución importante en el desarrollo del proyecto, al incorporar mejoras que fortalecen la arquitectura, organización y calidad de la aplicación. La separación de los componentes permitió una estructura más flexible y preparada para futuras integraciones, mientras que los mecanismos de validación y pruebas implementados contribuyeron a mejorar la estabilidad y confiabilidad del sistema.

En conjunto, los avances alcanzados representan una base sólida para continuar con el desarrollo del proyecto, facilitando la incorporación de nuevas funcionalidades y la implementación de una solución más escalable y mantenible.

Limitaciones actuales de la solución.

Aunque el sistema ha evolucionado significativamente durante este módulo, aún existen componentes que requieren ser implementados e integrados por completo. Entre ellos se encuentran el almacenamiento persistente de la información, el despliegue de la aplicación en un entorno de producción y la optimización del sistema para soportar un mayor número de usuarios.

Consideraciones éticas, de seguridad y uso responsable de la IA.

El desarrollo del proyecto se realizó bajo principios de privacidad, seguridad y uso responsable de la inteligencia artificial. Se implementaron mecanismos de validación y control para reducir el riesgo de respuestas inadecuadas y garantizar un procesamiento confiable de la información. Asimismo, la arquitectura propuesta prioriza la protección de los datos de los usuarios, procurando que el tratamiento de la información se realice de forma segura y conforme a las necesidades del proyecto.

Conclusiones

Las conclusiones derivadas del diseño, desarrollo e implementación del sistema "Gerardito" reafirman la viabilidad y necesidad de modernizar los procesos de orientación vocacional en la educación superior. A partir del trabajo realizado, se establecen las siguientes conclusiones fundamentales:

Innovación en la Orientación Vocacional: Se demostró empíricamente que la transición de sistemas basados en reglas estáticas hacia arquitecturas de Inteligencia Artificial Generativa incrementa sustancialmente la precisión y empatía del diagnóstico. Al permitir entradas de texto libre y aplicar tolerancia a variaciones lingüísticas y ortográficas, el sistema elimina las barreras de los test tradicionales, adaptándose a la realidad comunicativa de los aspirantes.

Eficiencia de la Arquitectura Híbrida: En el ámbito técnico, la integración de un Modelo de Lenguaje Grande (Llama 3.1) con bibliotecas tradicionales de Procesamiento de Lenguaje Natural (spaCy) probó ser una solución altamente eficiente. Esta arquitectura permite delegar el razonamiento semántico complejo a la GPU, mientras que las tareas de extracción sintáctica se resuelven ágilmente en la CPU, optimizando los recursos de *hardware* bajo un enfoque práctico de MLOps y garantizando tiempos de respuesta mínimos.

Seguridad y Robustez del Sistema: La implementación de validadores semánticos garantizó la integridad estructural de la aplicación. Se comprobó que el uso de *Prompt Engineering* restrictivo es una medida de ciberseguridad efectiva para contener ataques de inyección (*prompt injection*), neutralizar lenguaje inapropiado y evitar alucinaciones por contexto, manteniendo al asistente estrictamente dentro de los límites de su dominio experto bajo el control de estado de Streamlit.

Ética y Soberanía de Datos: Finalmente, el despliegue de los modelos en un entorno estrictamente local reafirma el compromiso del proyecto con la privacidad por diseño. Se concluye que es plenamente factible integrar capacidades avanzadas de IA en la Universidad Gerardo Barrios sin comprometer los datos estudiantiles hacia servidores de terceros, sentando un precedente tecnológico, escalable y éticamente responsable para futuras innovaciones institucionales.

REFERENCIAS

Universidad Técnica de Machala- 2024 *Desarrollo de una aplicación de orientación vocacional para bachilleres mediante test utilizando inteligencia artificial para la toma de decisiones* <https://repositorio.utmachala.edu.ec/items/9c22ef84-fd35-44c3-ad6c-e7bd3bb66105>

Instituto Superior Universitario Rumiñahui - 2021 *Integrando tecnologías asociadas a data science en procesos de orientación vocacional y profesional* <https://revista.ister.edu.ec/ojs/index.php/ISTER/article/view/20>

Pérez-Díaz, M. - 2024 *Aplicación de inteligencia artificial en la educación de América Latina: tendencias, beneficios y desafíos* https://revistaveritas.org/index.php/veritas/article/view/52?utm_source=chatgpt.com

Universidad ORT Uruguay - 2025 *Inteligencia artificial en educación superior: revisión integrativa de la literatura* https://revistas.ort.edu.uy/cuadernos-de-investigacion-educativa/article/view/4034?utm_source=chatgpt.com